

Individuelle Praktische Arbeit

Sven Toye

Anforderungen

- Spieler:
 - id
 - first_name
 - last_name
 - gender (male, female, diverse)
- Trainer:
 - id
 - first_name
 - last_name
 - gender (male, female, diverse)
- Squad:
 - id
 - description
 - date
 - player (list of players + positions)
 - trainer (list of trainers)

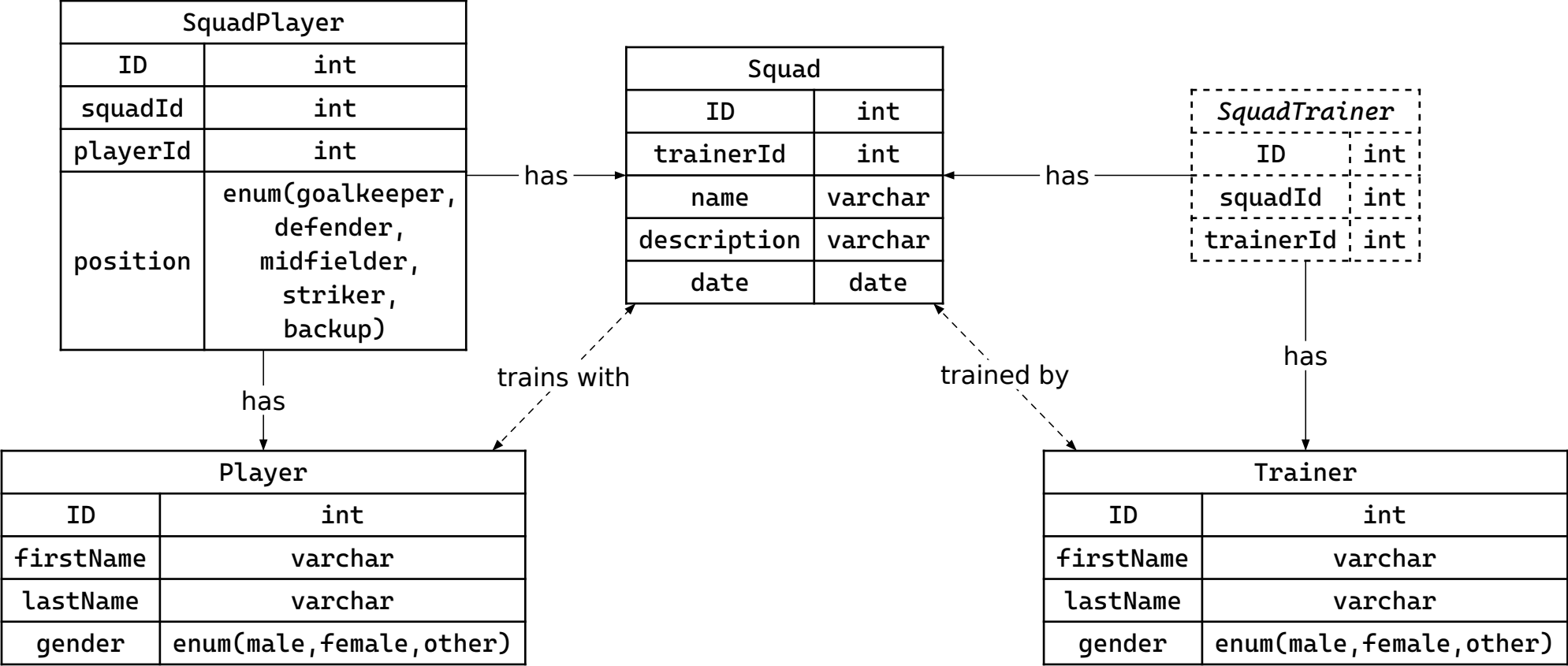
SquadPlayer	
ID	int
squadId	int
playerId	int
position	enum(goalkeeper, defender, midfielder, striker, backup)

Squad	
ID	int
trainerId	int
name	varchar
description	varchar
date	date

<i>SquadTrainer</i>	
ID	int
squadId	int
trainerId	int

Player	
ID	int
firstName	varchar
lastName	varchar
gender	enum(male, female, other)

Trainer	
ID	int
firstName	varchar
lastName	varchar
gender	enum(male, female, other)



Player

```
@SerializeOptions({ excludeExtraneousValues: true })
@Entity()
export class Player {
  @ApiProperty({ description: "ID", example: 1 })
  @PrimaryGeneratedColumn()
  id: number;

  @ApiProperty({ description: "Vorname", example: "Sven" })
  @Column({ length: 255 })
  firstName: string;

  @ApiProperty({ description: "Nachname", example: "Toye" })
  @Column({ length: 255 })
  lastName: string;

  @ApiProperty({ description: "Geschlecht" })
  @Column({ type: "enum", enum: Gender })
  gender: Gender;
}
```

```
enum Gender {
  Male = "Male",
  Female = "Female",
  Other = "Other",
}
```

Trainer

```
@SerializeOptions({ excludeExtraneousValues: true })
@Entity()
class Trainer {
    @ApiProperty({ description: "ID", example: 1 })
    @PrimaryGeneratedColumn()
    id: number;

    @ApiProperty({ description: "Vorname", example: "Sven" })
    @Column({ length: 255 })
    firstName: string;

    @ApiProperty({ description: "Nachname", example: "Toye" })
    @Column({ length: 255 })
    lastName: string;

    @ApiProperty({ description: "Geschlecht" })
    @Column({ type: "enum", enum: Gender })
    gender: Gender;
}
```

```
enum Gender {
    Male = "Male",
    Female = "Female",
    Other = "Other",
}
```

Squad

```
@SerializeOptions({ excludeExtraneousValues: true })
@Entity()
class Squad {
    @ApiProperty({ description: "ID", example: 1 })
    @PrimaryGeneratedColumn()
    id: number;

    @ApiProperty({ description: "Name", example: "FC Zürich" })
    @Column({ length: 255 })
    name: string;

    @ApiProperty({ description: "Beschreibung", example: "Seit 1896." })
    @Column({ length: 255 })
    description: string;

    @ApiProperty({ description: "Erstellungsdatum" })
    @Column()
    date: Date;
    ...
}
```

```
...
@ApiProperty({ description: "Trainer für diesen Squad" })
@ManyToMany(() => Trainer, {
    eager: true,
    cascade: true,
    nullable: false,
})
@JoinTable()
trainers: Trainer[];

@ApiProperty({ description: "Spieler mit ihren Positionen in diesem Team" })
@OneToMany(() => SquadPlayer, (squadPlayer) => squadPlayer.squad, {
    eager: true,
    cascade: true,
})
squadPlayers: SquadPlayer[];
}
```

SquadPlayer

```
@Entity()
class SquadPlayer {
    @ApiProperty({ description: "ID", example: 1 })
    @PrimaryGeneratedColumn()
    id: number;

    @ApiProperty({ description: "Position vom Spieler" })
    @Column({ type: "enum", enum: Position })
    position: Position;
    ...
}
```

```
enum Position {
    Goalkeeper = "Goalkeeper",
    Defender = "Defender",
    Midfielder = "Midfielder",
    Striker = "Striker",
    Backup = "Backup",
}
```



```
...  
@ApiModelProperty({ description: "Spieler" })  
@ManyToOne(() => Player, {  
    eager: true,  
    nullable: false,  
    orphanedRowAction: "delete",  
    onDelete: "CASCADE",  
    onUpdate: "CASCADE",  
})  
player: Player;  
  
@ApiHideProperty()  
@Exclude()  
@ManyToOne(() => Squad, (squad) => squad.squadPlayers, {  
    nullable: false,  
    orphanedRowAction: "delete",  
    onDelete: "CASCADE",  
    onUpdate: "CASCADE",  
})  
squad: Squad;  
}
```

Player DTO

```
@SerializeOptions({ excludeExtraneousValues: true })
class CreatePlayerDto {
    @ApiProperty({ description: "Vorname", example: "Sven" })
    @IsString()
    @IsNotEmpty()
    firstName: string;

    @ApiProperty({ description: "Nachname", example: "Toye" })
    @IsNotEmpty()
    @IsString()
    lastName: string;

    @ApiProperty({ description: "Geschlecht" })
    @IsEnum(Gender)
    gender: Gender;
}

class UpdatePlayerDto extends PartialType(CreatePlayerDto) {}
```

Trainer DTO

```
@SerializeOptions({ excludeExtraneousValues: true })
class CreateTrainerDto {
    @ApiProperty({ description: "Vorname", example: "Sven" })
    @IsString()
    @NotEmpty()
    firstName: string;

    @ApiProperty({ description: "Nachname", example: "Toye" })
    @IsString()
    @NotEmpty()
    lastName: string;

    @ApiProperty({ description: "Geschlecht" })
    @IsEnum(Gender)
    gender: Gender;
}

class UpdateTrainerDto extends PartialType(CreateTrainerDto) {}
```

Squad DTO

```
@SerializeOptions({ excludeExtraneousValues: true })
export class CreateSquadDto {
  @ApiProperty({ description: "Name", example: "FC Zürich" })
  @IsString()
  @IsNotEmpty()
  name: string;

  @ApiProperty({ description: "Beschreibung", example: "Seit 1896." })
  @IsString()
  description: string;

  @ApiProperty({ description: "Erstellungsdatum" })
  @IsDate()
  @Type(() => Date)
  date: Date;
  ...
}
```

```
...  
@ApiProperty({ description: "Trainer für diesen Squad" })  
@ValidateNested()  
@IsInstance(TrainerId, { each: true })  
@Type(() => TrainerId)  
trainers: TrainerId[];  
  
@ApiProperty({ description: "Spieler mit ihren Positionen in diesem Team" })  
@ValidateNested()  
@IsInstance(SquadSquadPlayerDto, { each: true })  
@Type(() => SquadSquadPlayerDto)  
squadPlayers: SquadSquadPlayerDto[];  
}  
  
@SerializeOptions({ excludeExtraneousValues: true })  
class TrainerId {  
    @ApiProperty({ description: "ID", example: 1 })  
    @IsInt()  
    @IsPositive()  
    id: number;  
}
```

SquadPlayer DTO (im Squad DTO)

```
@SerializeOptions({ excludeExtraneousValues: true })
class SquadSquadPlayerDto {
    @ApiProperty({ description: "Spieler" })
    @ValidateNested()
    @IsInstance(PlayerId)
    @Type(() => PlayerId)
    player: PlayerId;

    @ApiProperty({ description: "Position vom Spieler" })
    position: Position;
}
```

```
@SerializeOptions({ excludeExtraneousValues: true })
class PlayerId {
    @ApiProperty({ description: "ID", example: 1 })
    @IsInt()
    @IsPositive()
    id: number;
}
```

Player Service

```
@Injectable()
```

```
export class PlayerService {
```

```
  constructor(
```

```
    @InjectRepository(Player)
```

```
    private playerRepository: Repository<Player>,
```

```
  ) {}
```

```
  create(createPlayerDto: CreatePlayerDto): Promise<Player> {
```

```
    return this.playerRepository.save(createPlayerDto);
```

```
  }
```

```
  findAll(): Promise<Player[]> {
```

```
    return this.playerRepository.find();
```

```
  }
```

```
  findOne(id: number): Promise<Player | null> {
```

```
    return this.playerRepository.findOneBy({ id });
```

```
  }
```

```
  ...
```

```
...
async update(
  id: number,
  updatePlayerDto: UpdatePlayerDto,
): Promise<Player | null> {
  const player = await this.findOne(id);
  if (!player) return null;
  const updatePlayer = this.playerRepository.merge(player, updatePlayerDto);
  return await this.playerRepository.save(updatePlayer);
}

async remove(id: number): Promise<boolean> {
  const exists = await this.playerRepository.existsBy({ id });
  if (exists) {
    await this.playerRepository.delete(id);
  }
  return exists;
}
}
```


Trainer Service

```
@Injectable()
export class TrainerService {
  constructor(
    @InjectRepository(Trainer)
    private trainerRepository: Repository<Trainer>,
  ) {}

  create(createTrainerDto: CreateTrainerDto): Promise<Trainer> {
    return this.trainerRepository.save(createTrainerDto);
  }

  findAll(): Promise<Trainer[]> {
    return this.trainerRepository.find();
  }

  findOne(id: number): Promise<Trainer | null> {
    return this.trainerRepository.findOneBy({ id });
  }

  ...
}
```

```
...
async update(
  id: number,
  updateTrainerDto: UpdateTrainerDto,
): Promise<Trainer | null> {
  const trainer = await this.findOne(id);
  if (!trainer) return null;
  const updateTrainer = this.trainerRepository.merge(trainer, updateTrainerDto);
  return await this.trainerRepository.save(updateTrainer);
}

async remove(id: number): Promise<boolean> {
  const exists = await this.trainerRepository.existsBy({ id });
  if (exists) {
    await this.trainerRepository.delete(id);
  }
  return exists;
}
}
```

Squad Service

```
@Injectable()
export class SquadService {
  constructor(
    @InjectRepository(Squad)
    private squadRepository: Repository<Squad>,
  ) {}

  async create(createSquadDto: CreateSquadDto): Promise<Squad> {
    return Squad.new(await this.squadRepository.save(createSquadDto));
  }

  findAll(): Promise<Squad[]> {
    return this.squadRepository.find();
  }

  findOne(id: number): Promise<Squad | null> {
    return this.squadRepository.findOneBy({ id });
  }

  async update(
    id: number,
```

```
    updateSquadDto: UpdateSquadDto,  
  ): Promise<Squad | null> {  
    const squad = await this.findOne(id);  
    if (!squad) return null;  
    const updateSquad: DeepPartial<Squad> = { ...squad, ...updateSquadDto };  
    return Squad.new(await this.squadRepository.save(updateSquad));  
  }  
  
  async remove(id: number): Promise<boolean> {  
    const exists = await this.squadRepository.existsBy({ id });  
    if (exists) {  
      await this.squadRepository.delete(id);  
    }  
    return exists;  
  }  
}
```

Anforderungen

- Player:

Method	Pfad	Privat?	Beschreibung
GET	/api/player/{id}	protected	Auslesen einzelner und aller Spieler
POST	/api/player	protected	Anlegen einzelner Spieler*innen
PUT	/api/player/{id}	protected	Änderung einer bestehend Spieler*in
DELETE	/api/player/{id}	protected	Löschen einer Spieler*in

- Trainer:

Method	Pfad	Privat?	Beschreibung
GET	/api/trainer/{id}	protected	Auslesen einzelner und aller Trainer*innen
POST	/api/trainer	protected	Anlegen einzelner Trainer*innen
PUT	/api/trainer/{id}	protected	Änderung einer bestehend Trainer*in
DELETE	/api/trainer/{id}	protected	Löschen einer Trainer*in

Anforderungen

- Squad:

Method	Pfad	Privat?	Beschreibung
GET	/api/squad/{id}	public	Einzelner Teamkader
POST	/api/squad	protected	Neuen Squad anlegen
PUT	/api/squad/{id}	protected	Squad bearbeiten
DELETE	/api/squad/{id}	protected	Squad löschen
GET	/api/squad	protected	Liste aller existierenden Teamkader

Protected Endpoints sind mit einem JSON Web Token geschützt und sind nur für Benutzer, die sich bei /api/login eingeloggt haben, zugänglich.

Player Controller

```
@ApiUnauthorizedResponse()
@ApiBearerAuth()
@UseGuards(JwtGuard)
@UseInterceptors(ClassSerializerInterceptor)
@Controller("/player")
class PlayerController {
    constructor(private readonly playerService: PlayerService) {}

    @ApiOperation({ summary: "Erstelle einen Spieler" })
    @ApiResponse({ type: Player, description: "Spieler erstellt" })
    @Post()
    create(@Body() createPlayerDto: CreatePlayerDto): Promise<Player> {
        return this.playerService.create(createPlayerDto);
    }

    @ApiOperation({ summary: "Lese alle Spieler aus" })
    @ApiResponse({ type: [Player], description: "Alle Spieler ausgelesen" })
    @Get()
    findAll(): Promise<Player[]> {
        return this.playerService.findAll();
    }
}
```

```
@ApiOperation({ summary: "Lese einen Spieler aus" })
@ApiOkResponse({ type: Player, description: "Spieler ausgelesen" })
@ApiNotFoundResponse({ description: "Spieler existiert nicht" })
@Get(":id")
async findOne(@Param("id") id: number): Promise<Player> {
  const player = await this.playerService.findOne(id);
  if (!player) throw new NotFoundException();
  return player;
}

@ApiOperation({ summary: "Bearbeite einen Spieler" })
@ApiOkResponse({ type: Player, description: "Spieler bearbeitet" })
@ApiNotFoundResponse({ description: "Spieler existiert nicht" })
@Patch(":id")
async update(
  @Param("id") id: number,
  @Body() updatePlayerDto: UpdatePlayerDto,
): Promise<Player> {
  const player = await this.playerService.update(id, updatePlayerDto);
  if (!player) throw new NotFoundException();
  return player;
}
```



```
@ApiOperation({ summary: "Lösche einen Spieler" })
@ApiOkResponse({ description: "Spieler gelöscht" })
@ApiNotFoundResponse({ description: "Spieler existiert nicht" })
@Delete(":id")
async remove(@Param("id") id: number): Promise<void> {
  const exists = await this.playerService.remove(id);
  if (!exists) throw new NotFoundException();
}
}
```

Trainer Controller

```
@ApiUnauthorizedResponse()
@ApiBearerAuth()
@UseGuards(JwtGuard)
@UseInterceptors(ClassSerializerInterceptor)
@Controller("/trainer")
export class TrainerController {
    constructor(private readonly trainerService: TrainerService) {}

    @ApiOperation({ summary: "Erstelle einen Trainer" })
    @ApiResponse({ type: Trainer, description: "Trainer erstellt" })
    @Post()
    create(@Body() createTrainerDto: CreateTrainerDto): Promise<Trainer> {
        return this.trainerService.create(createTrainerDto);
    }

    @ApiOperation({ summary: "Lese alle Trainer aus" })
    @ApiOkResponse({ type: [Trainer], description: "Alle Trainer ausgelesen" })
    @Get()
    findAll(): Promise<Trainer[]> {
        return this.trainerService.findAll();
    }
}
```

```
@ApiOperation({ summary: "Lese einen Trainer aus" })
@ApiOkResponse({ type: Trainer, description: "Trainer ausgelesen" })
@ApiNotFoundResponse({ description: "Trainer existiert nicht" })
@Get(":id")
async findOne(@Param("id") id: number): Promise<Trainer> {
  const trainer = await this.trainerService.findOne(id);
  if (!trainer) throw new NotFoundException();
  return trainer;
}

@ApiOperation({ summary: "Bearbeite einen Trainer" })
@ApiOkResponse({ type: Trainer, description: "Trainer bearbeitet" })
@ApiNotFoundResponse({ description: "Trainer existiert nicht" })
@Patch(":id")
async update(
  @Param("id") id: number,
  @Body() updateTrainerDto: UpdateTrainerDto,
): Promise<Trainer> {
  const trainer = await this.trainerService.update(id, updateTrainerDto);
  if (!trainer) throw new NotFoundException();
  return trainer;
}
```

```
@ApiOperation({ summary: "Lösche einen Trainer" })
@ApiOkResponse({ description: "Trainer gelöscht" })
@ApiNotFoundResponse({ description: "Trainer existiert nicht" })
@Delete(":id")
async remove(@Param("id") id: number): Promise<void> {
  const exists = await this.trainerService.remove(id);
  if (!exists) throw new NotFoundException();
}
}
```

Squad Controller

```
@Controller("/squad")
@UseInterceptors(ClassSerializerInterceptor)
export class SquadController {
    constructor(private readonly squadService: SquadService) {}

    @ApiOperation({ summary: "Erstelle einen Squad" })
    @ApiResponse({ type: Squad, description: "Squad erstellt" })
    @ApiUnauthorizedResponse() @ApiBearerAuth() @UseGuards(JwtGuard)
    @Post()
    create(@Body() createSquadDto: CreateSquadDto): Promise<Squad> {
        return this.squadService.create(createSquadDto);
    }

    @ApiOperation({ summary: "Lese alle Squad aus" })
    @ApiOkResponse({ type: [Squad], description: "Alle Squads ausgelesen" })
    @ApiUnauthorizedResponse() @ApiBearerAuth() @UseGuards(JwtGuard)
    @Get()
    findAll(): Promise<Squad[]> {
        return this.squadService.findAll();
    }
}
```

```
@ApiOperation({ summary: "Lese einen Squad aus" })
@ApiOkResponse({ type: Squad, description: "Squad ausgelesen" })
@ApiNotFoundResponse({ description: "Squad existiert nicht" })
@Get(":id")
async findOne(@Param("id") id: number): Promise<Squad | null> {
    const squad = await this.squadService.findOne(id);
    if (!squad) throw new NotFoundException();
    return squad;
}

@ApiOperation({ summary: "Bearbeite einen Squad" })
@ApiOkResponse({ type: Squad, description: "Squad bearbeitet" })
@ApiNotFoundResponse({ description: "Squad existiert nicht" })
@ApiUnauthorizedResponse() @ApiBearerAuth() @UseGuards(JwtGuard)
@Patch(":id")
async update(
    @Param("id") id: number,
    @Body() updateSquadDto: UpdateSquadDto,
): Promise<Squad | null> {
    const squad = await this.squadService.update(id, updateSquadDto);
    if (!squad) throw new NotFoundException();
    return squad;
}
```

```
@ApiOperation({ summary: "Lösche einen Squad" })
@ApiOkResponse({ description: "Squad gelöscht" })
@ApiNotFoundResponse({ description: "Squad existiert nicht" })
@ApiUnauthorizedResponse()
@ApiBearerAuth()
@UseGuards(JwtGuard)
@Delete(":id")
async remove(@Param("id") id: number): Promise<void> {
    const exists = await this.squadService.remove(id);
    if (!exists) throw new NotFoundException();
}
}
```